

NUMA Memory Policy for Fusion Compiler-Class Jobs

Ops / design note for NUMA-EDA-Bench

Scope: Bind CPU and DRAM to the same NUMA node for one fat PD job when the working set fits that node's free memory.

Sibling: PD Job Acceleration (tmpfs + rsync) addresses filesystem placement; this note addresses DRAM placement.

Abstract

Multi-socket Linux hosts expose NUMA: each socket owns cores and DRAM. Cross-socket loads pay interconnect latency and bandwidth tax. Physical-design tools with large heaps are exposed to **first-touch** placement and scheduler migration, which can leave threads on one node and pages on another while top still reports busy CPUs.

The operational response—when peak RSS fits one node's free RAM—is:

```
numactl --cpunodebind=N --membind=N <eda_command>
```

This repository provides a topology report, a production wrapper with a MemFree guard, and a **hardware-only** microbench compare (remote membind vs local membind). Single-node hosts cannot measure remote DRAM; the compare script refuses to emit a fake speedup on those machines.

1. Problem

Accelerate turnaround of a PD tool on a multi-socket host by aligning CPU affinity and memory binding to one NUMA node, without changing the tool binary or functional results, and without hard-binding into OOM.

Success:

1. Same Tcl / same deliverables.
 2. On ≥ 2 -node hardware: measurable triad $\uparrow$ and/or chase latency $\downarrow$ for local vs remote policy (microbench), and/or better stage wall time with healthier numastat.
 3. Wrapper refuses unsafe membind when MemFree is too low (unless overridden).
-

2. NUMA and first-touch

- **Local DRAM** — CPU and page on the same node.

- **Remote DRAM** — page on the other node; traffic crosses UPI / Infinity Fabric.
- **First-touch** — Linux often places a newly written anonymous page on the node of the CPU that first wrote it. Master-thread init + workers elsewhere is a classic remote-fill pattern.

Discovery:

```
./scripts/numa_report.sh
# or: numactl -H
```

3. Policy

Flag	Meaning
--cpunodebind=N	run on node N CPUs only
--membind=N	allocate only from node N (hard)
--preferred=N	prefer node N, spill if needed (softer)

Recipe for one job that fits node 0:

```
./scripts/run_eda_numactl.sh 0 fc_shell -f run.tcl
```

Use POLICY=preferred when peak RSS is uncertain.

4. Implementation in this repo

Component	Role
scripts/numa_report.sh	topology + MemFree + recommendation
scripts/run_eda_numactl.sh	wrap tool; MemFree guard; membind/preferred
examples/compare_numa.sh	hardware remote vs local microbench (median)
src/numa_mem_bench.cpp	STREAM triad + pointer chase
tools/numactl-root/	vendored numactl / numastat

Compare exits with code 2 on single-node hosts. It does not synthesize remote costs.

5. Measurement

5.1 Microbench metrics

Name	Definition	Better
triad	STREAM $a[i]=b[i]+s*c[i]$ GiB/s	higher

Name	Definition	Better
chase	shuffled pointer-chase ns/hop	lower

Median of several trials; spreads are printed.

5.2 What is citeable

Situation	Cite?
compare_numa.sh on ≥ 2 nodes (CLAIM_GATE=CITEABLE=yes)	Yes — as microbench under numactl , not as FC wall time
1-node / cloud VM	No — script refuses
Real FC stage wall time + numastat -p	Yes — strongest farm claim

5.3 Results in this CI / cloud environment

This agent VM has **1 NUMA node**. There is **no hardware remote-vs-local result to report**. That is correct behavior, not a missing table.

Re-run on farm silicon and attach SUMMARY.txt when you have it.

6. When it helps / when it does not

Situation	Expectation
2S/4S, one FC, RSS fits node free RAM	Primary target
1 NUMA node	No locality win from membind
RSS > node free RAM + hard membind	Harm (OOM / reclaim)
Bound is NFS I/O	Fix storage tiering first
Many small jobs	One heavy job per node; don't pile onto one socket blindly

7. Safety

1. Check MemFree per node before hard membind.
 2. Default wrapper floor: 8 GiB free (MIN_MEMFREE_GIB).
 3. Prefer POLICY=preferred when unsure.
 4. Log numactl -H into the run record.
 5. During run: numastat -p <pid>.
-

8. Review defense

1. Show topology (numactl -H).
 2. State hypothesis (remote fills / first-touch).
 3. Show hardware microbench local vs remote **or** stage wall time + numastat.
 4. Show MemFree vs peak RSS (safety).
 5. Do not claim single-node or emulated figures.
-

9. Conclusion

NUMA binding is a placement policy, not a PD algorithm. It is useful when the machine is multi-socket and the job fits one node. This toolkit is written to be **usable on a farm** and **honest in CI**: no fabricated remote-DRAM speedups on single-node hosts.

Appendix: commands

```
./scripts/numa_report.sh
./scripts/run_eda_numactl.sh 0 fc_shell -f run.tcl
make -j && TRIALS=5 BYTES=1G ./examples/compare_numa.sh
python3 docs/build_whitepaper_pdf.py
```